

Service Catalog

More information about the service catalog can be found in the [Service Inventory Catalog](#) page.

The `stage-group-mapping.jsonnet` file is generated from `stages.yml` in the handbook by running `scripts/update-stage-groups-feature-categories`.

The `stage-group-mapping-overrides.jsonnet` file can be updated manually in order to transform one stage group into another, or merge two or more stage groups into one. The feature categories of the selected groups will be associated with the new group. All the groups being merged need to belong to the same stage.

Definition of a Service

A **service** is a logical operational unit that provides a specific business or technical function, has clear ownership, and typically requires monitoring and incident response capabilities. A service can be described in terms of what it does, not where it runs.

Core Characteristics

A service in the GitLab Infrastructure Service Catalog **MUST** have:

1. **Continuous Operation** - Runs as a process, scheduled job, or is triggered automatically (not manually executed by humans)
2. **Clear Ownership** - Attributable to a specific team responsible for its operation, cost, maintenance and evolution

A service in the GitLab Infrastructure Service Catalog **SHOULD** have:

1. **Observability** - Monitoring, dashboards, SLIs, SLOs, and alerting
2. **Incident Response Responsibilities** - Requires on-call coverage and has associated runbooks
3. **Independent Lifecycle** - Can be deployed, scaled, and failed independently of other services

Service Scope

Services can exist at different levels of abstraction:

Category	Description	Examples
Application Services	User-facing or business logic services	<code>web</code> , <code>api</code> , <code>registry</code> , <code>sidekiq</code>
Platform Services	Infrastructure services with APIs consumed by applications	<code>patroni</code> , <code>redis-*</code> , <code>gitaly</code>

Category	Description	Examples
Supporting Services	Discrete microservices providing specific capabilities	<code>ai-gateway</code> , <code>kas</code> , <code>spamcheck</code>
External Dependencies	Third-party SaaS integrations we depend on	<code>cloudflare</code> , <code>mailgun</code>
Operational Services	Services we run that help us run the organization or the platform	<code>capacity_planning</code> , <code>error-budget-reporting</code>

Guiding Principles

- **Prefer inclusion over exclusion** - It's better to have an entry for something we can attribute ownership and observability to than to have no knowledge of a running component
- **Single-tenant internal instances** (like `dev-gitlab-org` or `ops-gitlab-net`) should be treated as a single service rather than broken down into constituent parts, as they don't serve customer traffic
- **Customer-facing platforms** (`GitLab.com`, `Dedicated`, `Cells`) should be broken down by individual services for granular ownership and cost attribution
- **Technical ownership** should remain with the team responsible for the service's codebase and architecture, regardless of where it runs
- **APIs between services** - As `GitLab` scales, well-defined interfaces between teams and services become increasingly important; the service catalog should reflect and encourage this

Teams.yml

The `teams.yml` file can contain a definition of a team responsible for a certain service or component (SLI). Possible configuration keys are:

- **product_stage_group**: The name of the stage group, if this team is a product stage group defined in `stages.yml`.
- **ignored_components**: If the team is a stage group, this key can be used to list components that should not feed into the stage group's error budget. The recordings for the group will continue for these components. But the component will not be included in error budgets in infradev reports, Sisense, or dashboards displaying the error budget for stage groups.
- **slack_alerts_channel**: The name of the Slack channel (without #) that the team would like to receive alerts in. Read more about alerting.
- **send_slo_alerts_to_team_slack_channel**: `true` or `false`. If the group would like to receive alerts for feature categories they own.

Schema

The service catalog adheres to JSON Schema specification for document annotation and validation. If you are interested in learning more, check out guides for getting started.

You can view the schema [here](#)

Modification

To modify the service catalog format, edit schema directly. Additional properties are disabled by default, please add new properties sparingly. For dynamic data, consider linking to single source of truth instead.

Right now, versioning is not required. To avoid breaking changes, consider only adding new properties in a backwards compatible manner similar to semantic versioning specification. If a property is no longer needed, please add `DEPRECATED:` prefix to `description` annotation.

Validation

The service catalog uses Ajv for schema validation and testing. During CI, tooling is used in `validate` and `test` stages. Here's an example failure:

```
[
  {
    "instancePath": "/services/0",
    "schemaPath": "#/definitions/ServiceDefinition/required",
    "keyword": "required",
    "params": {
      "missingProperty": "friendly_name"
    },
    "message": "must have required property 'friendly_name'"
  }
]
```

When a failure occurs, address any error messages and push up changes to re-run job.

Editor Support

One of the benefits of JSON Schema is **optional support** for multiple editors. If your preferred IDE is supported, follow setup instructions and edit `service-catalog.yml` and/or `teams.yml` as you normally would.

After successful setup, the developer experience should be greatly improved with features such as code completion for properties, hover for annotations, and highlighting for validations.

Service Labels

For each entry in the service catalog, a label is automatically created by the `reconcile_service_catalog_labels` job in CI when a service label does not already exist.

The label name can be customized using the `label` field in the service catalog, which will automatically convert to scoped label, e.g. `Service::my-service`.

The full list of service labels can be viewed under group labels.